COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	September 14, 2026	**DATE SUBMITTED:**	September 15, 2026
NAME:	Mendoza, Ryo Shin P.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 8.1 SORTING TECHNIQUES PART 2

1. PROCEDURE OUTPUT

ILO A: Create C++ code for implementation of shell, merge and quick sort.

Code + Console Screenshot

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgoprt2.h"

const int SIZE = 100;

// Function to display the array
void displayArray(int arr[], int size)
{
    for (int i = 0; i < size; i++)
    {
        std::cout << arr[i] << "\t";

        // Move to the next line every 10 values
        if ((i + 1) % 10 == 0)
        {
            std::cout << std::endl;
        }
    }
}

// Function to copy one array into another
void copyArray(int source[], int destination[], int size)
{
    for (int i = 0; i < size; i++)
    {
        destination[i] = source[i];
    }
}
```

```cpp
int main()
{
    // Create the original dataset
    int values[SIZE];

    // Create separate arrays for each sorting algorithm
    int shellValues[SIZE];
    int mergeValues[SIZE];
    int quickValues[SIZE];

    // Seed the random number generator
    std::srand(static_cast<unsigned int>(std::time(nullptr)));

    // Generate 100 random values from 1 to 99
    for (int i = 0; i < SIZE; i++)
    {
        values[i] = (std::rand() % 99) + 1;
    }

    // Copy the same dataset for each sorting algorithm
    copyArray(values, shellValues, SIZE);
    copyArray(values, mergeValues, SIZE);
    copyArray(values, quickValues, SIZE);

    // ========================
    // ORIGINAL ARRAY
    // ========================
    std::cout << "======================================\n";
    std::cout << "       ORIGINAL UNSORTED ARRAY\n";
    std::cout << "======================================\n";

    displayArray(values, SIZE);
```

```
======================================
       ORIGINAL UNSORTED ARRAY
======================================
40      65      72      87      5       95      45      37      80      16
90      52      77      99      5       35      73      18      19      19
40      86      38      87      73      23      49      89      45      19
14      6       77      54      89      7       66      70      12      61
39      38      73      26      74      20      25      79      86      74
51      33      85      96      40      77      17      44      79      98
5       48      26      29      83      46      79      61      55      53
84      82      63      85      8       67      95      76      14      97
29      86      90      24      26      48      35      40      85      51
61      9       72      97      31      91      10      54      82      10
```

Observations	The program generated 100 random values and displayed them using the displayArray() function. The copyArray() function was then used to create separate copies of the original dataset for Shell Sort, Merge Sort, and Quick Sort. This allows all three sorting algorithms to work with the same set of values while keeping the original array unchanged.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot

```cpp
// ========================
// SHELL SORT
// ========================
template <typename T>
void shellSort(T arr[], int size)
{
    // Start with half of the array as the gap
    for (int gap = size / 2; gap > 0; gap /= 2)
    {
        // Perform insertion sort using the gap
        for (int i = gap; i < size; i++)
        {
            T temp = arr[i];
            int j;

            // Move elements that are greater than temp
            // to the right
            for (j = i; j >= gap && arr[j - gap] > temp; j -= gap)
            {
                arr[j] = arr[j - gap];
            }

            // Place temp in its correct position
            arr[j] = temp;
        }
    }
}
```

```cpp
// ========================
// SHELL SORT
// ========================
std::cout << "\n=======================================\n";
std::cout << "             SHELL SORT\n";
std::cout << "=======================================\n";

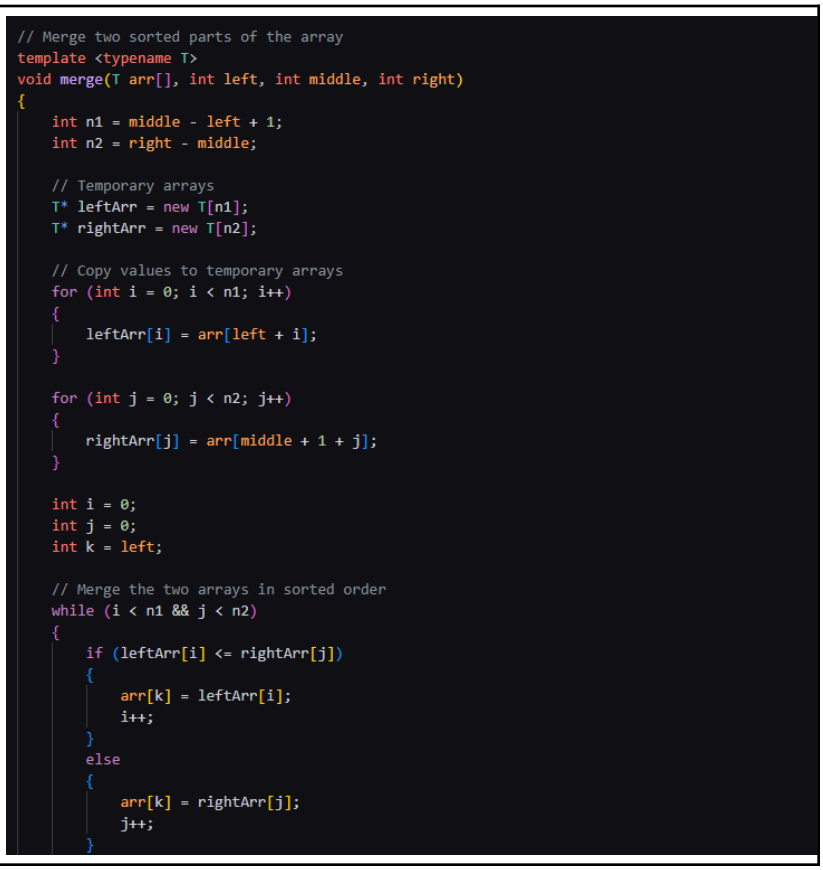
shellSort(shellValues, SIZE);

displayArray(shellValues, SIZE);
```

```
=======================================
            SHELL SORT
=======================================
1       2       3       5       5       6       6       6       6       7
7       8       9       9       10      11      12      12      12      13
14      15      16      16      17      18      19      20      21      21
22      22      27      27      27      28      30      30      32      35
35      36      36      37      41      41      41      41      44      46
48      48      49      50      52      54      54      55      56      57
58      62      63      64      65      66      67      69      69      71
75      76      76      77      77      78      79      79      79      80
83      83      83      84      86      86      90      90      90      91
91      92      93      95      95      97      98      98      99      99
```

Observations	The Shell Sort successfully sorted the 100 random values in ascending order. It used different gaps to compare and arrange elements that were far apart, making the array more organized before the final sorting pass.

Table 8-2. Shell Sort Technique

Code + Console Screenshot	

```
// Merge two sorted parts of the array
template <typename T>
void merge(T arr[], int left, int middle, int right)
{
    int n1 = middle - left + 1;
    int n2 = right - middle;

    // Temporary arrays
    T* leftArr = new T[n1];
    T* rightArr = new T[n2];

    // Copy values to temporary arrays
    for (int i = 0; i < n1; i++)
    {
        leftArr[i] = arr[left + i];
    }

    for (int j = 0; j < n2; j++)
    {
        rightArr[j] = arr[middle + 1 + j];
    }

    int i = 0;
    int j = 0;
    int k = left;

    // Merge the two arrays in sorted order
    while (i < n1 && j < n2)
    {
        if (leftArr[i] <= rightArr[j])
        {
            arr[k] = leftArr[i];
            i++;
        }
        else
        {
            arr[k] = rightArr[j];
            j++;
        }
```

```
        k++;
    }

    // Copy remaining elements from left array
    while (i < n1)
    {
        arr[k] = leftArr[i];
        i++;
        k++;
    }

    // Copy remaining elements from right array
    while (j < n2)
    {
        arr[k] = rightArr[j];
        j++;
        k++;
    }

    delete[] leftArr;
    delete[] rightArr;
}
```

```
// Merge Sort function
template <typename T>
void mergeSort(T arr[], int left, int right)
{
    // Continue dividing while there is more than one element
    if (left < right)
    {
        int middle = left + (right - left) / 2;

        // Sort the left half
        mergeSort(arr, left, middle);

        // Sort the right half
        mergeSort(arr, middle + 1, right);

        // Merge the sorted halves
        merge(arr, left, middle, right);
    }
}
```

```
std::cout << "\n=======================================\n";
std::cout << "             MERGE SORT\n";
std::cout << "=======================================\n";

mergeSort(mergeValues, 0, SIZE - 1);

displayArray(mergeValues, SIZE);
```

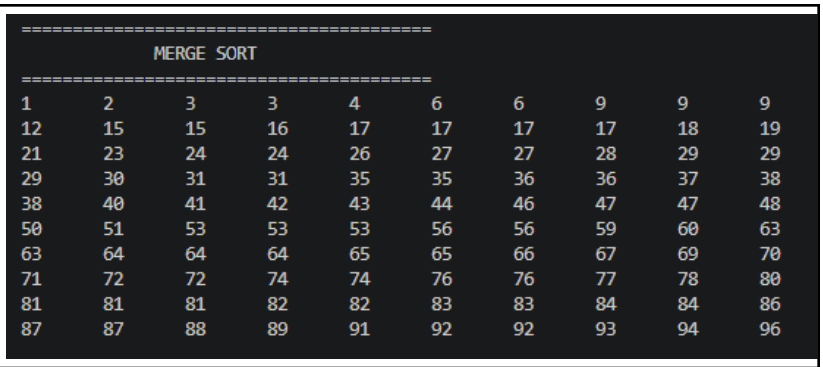
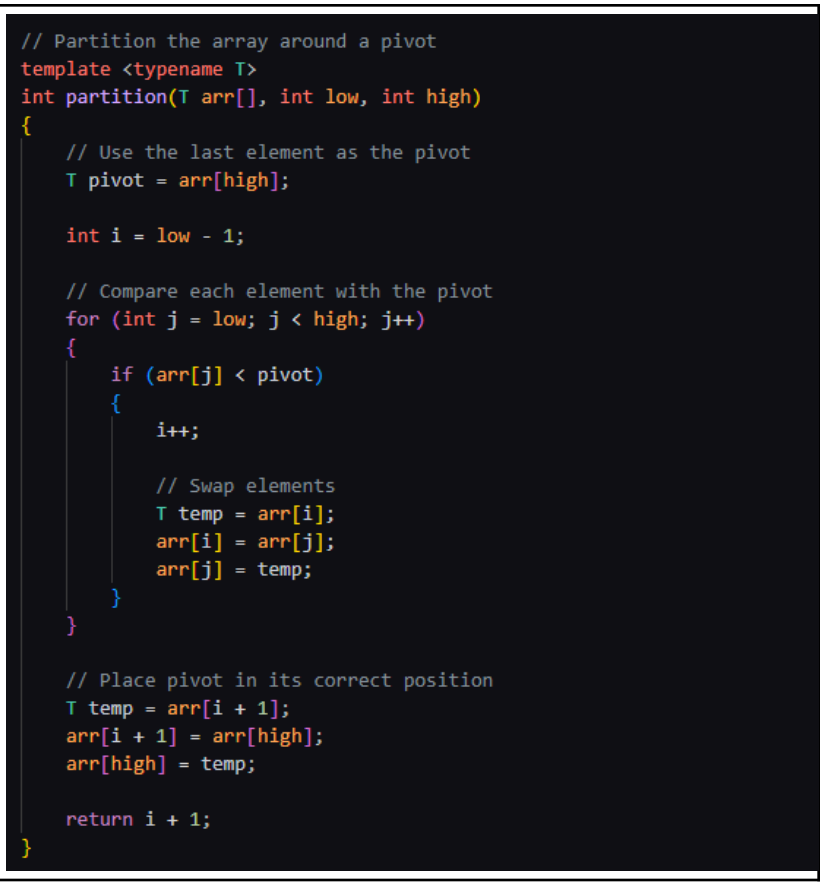
	
Observations	The Merge Sort successfully sorted the 100 random values in ascending order. The array was divided into smaller sub-arrays, each part was sorted, and the sorted sub-arrays were merged to form one complete sorted array.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	

```
// Partition the array around a pivot
template <typename T>
int partition(T arr[], int low, int high)
{
    // Use the last element as the pivot
    T pivot = arr[high];

    int i = low - 1;

    // Compare each element with the pivot
    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;

            // Swap elements
            T temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    // Place pivot in its correct position
    T temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}
```

```
// Quick Sort function
template <typename T>
void quickSort(T arr[], int low, int high)
{
    if (low < high)
    {
        // Get the pivot position
        int pivot = partition(arr, low, high);

        // Sort the left side
        quickSort(arr, low, pivot - 1);

        // Sort the right side
        quickSort(arr, pivot + 1, high);
    }
}

#endif
```

```
// ========================
// QUICK SORT
// ========================
std::cout << "\n========================================\n";
std::cout << "             QUICK SORT\n";
std::cout << "========================================\n";

quickSort(quickValues, 0, SIZE - 1);

displayArray(quickValues, SIZE);
```

```
====================================
            QUICK SORT
====================================
1       2       3       3       4       6       6       9       9       9
12      15      15      16      17      17      17      17      18      19
21      23      24      24      26      27      27      28      29      29
29      30      31      31      35      35      36      36      37      38
38      40      41      42      43      44      46      47      47      48
50      51      53      53      53      56      56      59      60      63
63      64      64      64      65      65      66      67      69      70
71      72      72      74      74      76      76      77      78      80
81      81      81      82      82      83      83      84      84      86
87      87      88      89      91      92      92      93      94      96
```

Observations	The Quick Sort successfully sorted the 100 random values in ascending order. It selected a pivot to partition the array into smaller parts and then sorted the left and right sub-arrays recursively until the entire array was sorted.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Answer:

Yes. The left and right sub-lists from Quick Sort can be sorted using other algorithms such as Insertion Sort, Bubble Sort, or Selection Sort.

- For example, after partitioning: **{8,3,7,2,5,1,6}**
- Using 5 as the pivot, we get: **{3,2,1} | 5 | {8,7,6}**
- Applying **Insertion Sort** to both sides produces: **{1,2,3} | 5 | {6,7,8}**
- Final Sorted Array: **{1, 2, 3, 5, 6, 7, 8}**

Overall, other sorting algorithms can be used to sort the left and right sub-lists after partitioning. This demonstrates that different sorting techniques can be combined to complete the sorting process.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

Answer:

Among the sorting algorithms discussed in this activity, Quick Sort would generally give the fastest time performance for this type of dataset when the pivot divides the array reasonably well. Its average time complexity is O(N log N), making it efficient for larger datasets.

Merge Sort has O(N log N) because it repeatedly divides the array into smaller parts. The array has about log N levels of division, and at each level, the elements are processed during merging, giving approximately N × log N operations.

Quick Sort can also have O(N log N) when the pivot divides the array into relatively balanced left and right sub-lists. Each partition processes about N elements, while balanced recursive divisions create about log N levels, resulting in O(N log N).

Conclusion
In this activity, I learned how Shell Sort, Merge Sort, and Quick Sort work and how they use different methods to sort data. I was able to implement and test them using 100 random values and understand their basic processes. The supplementary activity also helped me understand Quick Sort partitioning and the O(N log N) time complexity of Merge Sort and Quick Sort. Overall, I think I did well, but I still need to improve my understanding of how each algorithm works step by step.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Assistance, grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used AI for code assistance and grammar correction since the activity was hard.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.